ASSIGNMENT-13.3

Name: Chirra Jyothika

Ht.no:2303A51280

Batch:05

Task 1: Refactoring – Removing Code Duplication

Task Description: Use AI assistance to refactor a legacy Python script that contains repeated

blocks of code calculating the area and perimeter of rectangles.

Starter (Legacy) Code

```
# Legacy script with repeated logic

print("Area of Rectangle:", 5 * 10)

print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)

print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)

print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Prompt:

J Refactor the following legacy Python script to remove duplicated code by creating reusable functions. Ensure the solution: Uses a function to calculate both area and perimeter of a rectangle Avoids repeated logic Includes proper docstrings for all functions Keeps the code clean and readable Legacy Code: print("Area of Rectangle:", 5 * 10) print("Perimeter of Rectangle:", 2 * (5 + 10)) print("Area of Rectangle:", 7 * 12) print("Perimeter of Rectangle:", 2 * (7 + 12)) print("Area of Rectangle:", 10 * 15) print("Perimeter of Rectangle:", 2 * (10 + 15))

Code:

```python
def calculate_rectangle_properties(width, height):
    """
    Calculates the area and perimeter of a rectangle.

    Args:
        width (int or float): The width of the rectangle.
        height (int or float): The height of the rectangle.

    Returns:
        tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = width * height
    perimeter = 2 * (width + height)
    return area, perimeter

# Refactored code using the new function
print("Calculating properties for width=5, height=10:")
area1, perimeter1 = calculate_rectangle_properties(5, 10)
print(f"Area of Rectangle: {area1}")
print(f"Perimeter of Rectangle: {perimeter1}\n")

print("Calculating properties for width=7, height=12:")
area2, perimeter2 = calculate_rectangle_properties(7, 12)
print(f"Area of Rectangle: {area2}")
print(f"Perimeter of Rectangle: {perimeter2}\n")

print("Calculating properties for width=10, height=15:")
area3, perimeter3 = calculate_rectangle_properties(10, 15)
print(f"Area of Rectangle: {area3}")
print(f"Perimeter of Rectangle: {perimeter3}")
```

Output:

```
Calculating properties for width=5, height=10:
Area of Rectangle: 50
Perimeter of Rectangle: 30

Calculating properties for width=7, height=12:
Area of Rectangle: 84
Perimeter of Rectangle: 38

Calculating properties for width=10, height=15:
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Explanation:

1. Original Problem: The old code repeated the same calculations for area and perimeter each time, which made it longer and harder to manage.
2. Solution: We created a special block of code called a function (named calculate_rectangle_properties). This function takes the width and height as input.

3. How it Works: Inside this function, the area and perimeter are calculated just once. Then, the function gives back both results.
4. Benefit: Now, instead of writing the calculation multiple times, we just 'call' this function with different widths and heights, making the code much cleaner and more efficient.

Task 2: Refactoring – Optimizing Loops and Conditionals

Task Description: Use AI to analyze and refactor a script that checks the presence of elements

using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]

search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:

found = False

for n in names:

if s == n:

found = True

if found:

print(f"{s} is in the list")

else:

print(f"{s} is not in the list")
```

Prompt:

J Refactor the following Python script to improve performance by eliminating inefficient nested loops. Optimize the solution using set lookups or Python comprehensions. Ensure the solution: Avoids nested loops where possible Uses efficient data structures (like sets) Includes clean, readable code Provides a brief performance comparison (before vs after refactoring) Legacy Code: names = ["Alice", "Bob", "Charlie", "David"] search_names = ["Charlie", "Eve", "Bob"] for s in search_names: found = False for n in names: if s == n: found = True if found: print(f"{s} is in the list") else: print(f"{s} is not in the list")

Code:

```python
import timeit

def legacy_search(names_list, search_items):
    """
    Performs a search using inefficient nested loops.

    Args:
        names_list (list): The list to search within.
        search_items (list): The items to search for.

    Returns:
        list: A list of messages indicating whether each item was found.
    """
    results = []
    for s in search_items:
        found = False
        for n in names_list:
            if s == n:
                found = True
                break # Found, no need to check further
        if found:
            results.append(f"{s} is in the list")
        else:
            results.append(f"{s} is not in the list")
    return results

def optimized_search(names_list, search_items):
    """
    Performs an optimized search using a set for efficient lookups.

    Args:
        names_list (list): The list to search within.
        search_items (list): The items to search for.

    Returns:
        list: A list of messages indicating whether each item was found.
    """
    # Convert the list to a set for O(1) average time complexity lookups
    names_set = set(names_list)
    results = []
    for s in search_items:
        if s in names_set:
            results.append(f"{s} is in the list")
        else:
            results.append(f"{s} is not in the list")
    return results

# Example usage with original data
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

print("--- Legacy Search Results ---")
legacy_results = legacy_search(names, search_names)
for r in legacy_results:
    print(r)

print("\n--- Optimized Search Results ---")
optimized_results = optimized_search(names, search_names)
for r in optimized_results:
    print(r)
```

Output:

```
--- Legacy Search Results ---
Charlie is in the list
Eve is not in the list
Bob is in the list

--- Optimized Search Results ---
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Explanation:

The original code uses nested loops, which repeatedly scan the entire list and make it slow.By converting the list to a set, we get constant-time lookups and avoid repeated comparisons.This reduces time

complexity from O(n×m) to O(n+m) and improves performance.The refactored code is also cleaner and easier to read.

Task 3: Refactoring – Extracting Reusable Functions

Task Description:

Refactor a legacy script where price and tax calculations are written inline.

Starter (Legacy) Code

```
price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)
```

Prompt:

J Refactor the following legacy Python script to improve modularity by extracting reusable functions. Ensure the solution: Creates a function calculate_total(price) to compute total price including tax Removes duplicated calculation logic Produces clean and reusable code Includes proper documentation (docstrings) Legacy Code: price = 250 tax = price * 0.18 total = price + tax print("Total Price:", total) price = 500 tax = price * 0.18 total = price + tax print("Total Price:", total)

Code:

```python
def calculate_total(price):
    """
    Calculates the total price including a fixed tax rate (18%).

    Args:
        price (float or int): The base price of an item.

    Returns:
        float: The total price after adding tax.
    """
    tax_rate = 0.18
    tax = price * tax_rate
    total = price + tax
    return total
 # Using the new calculate_total function

# First item
price1 = 250
total_price1 = calculate_total(price1)
print(f"Total Price for {price1}: {total_price1}")

# Second item
price2 = 500
total_price2 = calculate_total(price2)
print(f"Total Price for {price2}: {total_price2}")
```

Output:

```
Total Price for 250: 295.0
Total Price for 500: 590.0
```

Explanation:

The legacy code repeats the same tax and total price calculation for different prices.Refactoring extracts this logic into a reusable function, avoiding duplication.This makes the program cleaner, modular, and easier to maintain or update.It also improves readability and reusability of the code.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Task Description:

Use AI to identify hardcoded values and replace them with constants.

Starter (Legacy) Code

print("Area of Circle:", 3.14159 * (7 ** 2))

print("Circumference of Circle:", 2 * 3.14159 * 7)

Prompt:

J Refactor the following Python script by replacing hardcoded values (magic numbers) with named constants. Ensure the solution: Introduces meaningful constants such as PI and RADIUS Removes magic numbers from calculations Improves readability and maintainability Keeps the code clean and simple Legacy Code: print("Area of Circle:", 3.14159 * (7 ** 2)) print("Circumference of Circle:", 2 * 3.14159 * 7)

Code:

```
# Define named constants
PI = 3.14159
RADIUS = 7

# Calculate and print the area of the circle using constants
area = PI * (RADIUS ** 2)
print(f"Area of Circle: {area}")

# Calculate and print the circumference of the circle using constants
circumference = 2 * PI * RADIUS
print(f"Circumference of Circle: {circumference}")
```

Output:

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Explanation:

The original code uses hardcoded numbers, which makes it harder to understand and update.Replacing them with named constants improves readability and maintainability. Constants clearly describe the purpose of values used in calculations.Changes can be made easily without modifying multiple lines.

Task 5: Refactoring – Improving Variable Naming and Readability

Task Description:

Refactor a script with unclear variable names.

Starter (Legacy) Code

a = 10

b = 20

c = a * b / 2

print(c)

Prompt:

J Refactor the following Python script to improve readability by using descriptive variable names and meaningful comments. Ensure the solution: Replaces unclear variable names with descriptive ones Adds inline comments explaining the logic Keeps the output identical Maintains clean and readable code Legacy Code: a = 10 b = 20 c = a * b / 2 print(c)

Code:

```
# Define the base and height of the triangle
base = 10
height = 20

# Calculate the area of the triangle using the formula: (base * height) / 2
area_of_triangle = (base * height) / 2

# Print the calculated area
print(area_of_triangle)
```

Output:

```
100.0
```

Explanation:

The original code uses unclear variable names, making it hard to understand. Descriptive names explain the purpose of each value and

improve readability. Inline comments help others quickly understand the logic. The functionality and output remain unchanged.